Project 2: Creating and Working with Your Database

Group 27
Steven Chang, Eleni Lyberopoulos, Alice McMurray, Mujtaba Mirhasan

Relational Schema:

Product(product_id, description, name, amount_in_stock, price)

Toy(product_id, material, age_group, dimensions, manufacturer)

- product_id is a foreign key to Product

Movie(product_id, rating, duration, director, release_date)

- product_id is a foreign key to Product

Book(product_id, published_date, publisher, ISBN, page_count)

- product_id is a foreign key to Product

Order(order_id, order_timestamp, customer_id, payment_id, shipping_id)

- customer_id is a foreign key to Customer
- payment_id is a foreign key to Payment
- shipping_id is a foreign key to Shipping

Payment(payment_id, payment_timestamp, payment_method, transaction_token, amount, last_four_digits)

Shipping(shipping_id, from_address, to_address, tracking_number, carrier_name, status, expected_delivery)

Customer(customer_id, first_name, last_name, email, password)

Author(author_id, author_name, biography)

Genre(genre_name)

Return(return_timestamp, order_id, product_id, quantity, shipping_id)

- (order_id, product_id) is a foreign key to OrderItem
- shipping_id is a foreign key to Shipping

OrderItem(order_id, product_id, quantity, price_at_purchase)

- order_id is a foreign key to Order
- product_id is a foreign key to Product

BookAuthor(author_id, product_id)

- author_id is a foreign key to Author
- product_id is a foreign key to Product

BookGenre(product_id, genre_name)

- product_id is a foreign key to Book
- genre_name is a foreign key to Genre

MovieGenre(product_id, genre_name)

- product_id is a foreign key to Movie
- genre_name is a foreign key to Genre

Pending constraints:

We cannot express the participation constraint for Author (that all authors in our database must write at least one book in our database). We cannot express that returns must be done in 30 days, i.e., that the difference between the return date and order date is less than 30 days. We cannot express that the quantities of returns for an OrderItem must not exceed the quantity bought of this OrderItem. We cannot express that a Shipping entity must be connected to exactly one Order or Product (since participation constraints can only work to one other entity, not two). We cannot express the temporal restrictions, such as the return timestamp needing to be after the order timestamp, and the expected shipping date needing to be after the order date.
Moreover, we have constraints that many values need to be non-negative, such as the amount in stock, price, min and max age for toys, duration of movies, page count, payment amount, and quantities in orders. We also have attributes that need to be of a certain type. For example, movie ratings must be one of "G", "PG", "PG-13", "R", "NC-17", or NULL for not rated. Also, ISBNs need to have 13 digits separated by 4 hyphens and must be valid according to the check digit rules.

SQL Queries:

Query 1:

a) This query gets all the books written by a certain author, in this case 'Sophia Turner' of author id 5. It returns the names of the books and their product id, price, ISBN, publisher and page count, as well. It is ordered alphabetically by book name.

b)
```
SELECT P.name book_name, A.author_name, P.product_id, P.price,
    B.ISBN, B.publisher, B.page_count
FROM BookAuthor BA, Product P, Author A, Book B
WHERE BA.author_id = A.author_id AND BA.product_id = P.product_id
    AND B.product_id = P.product_id
    AND A.author_id = 5
ORDER BY book_name;
```

c) Screenshot: (Note: I used VARCHAR to make the columns thinner for the screenshot)

```
db2 => SELECT VARCHAR(P.name,15) book_name, VARCHAR(A.author_name,15) author_name, P.product_id, P.price,
    B.ISBN, VARCHAR(B.publisher,15) publisher, B.page_count
FROM BookAuthor BA, Product P, Author A, Book B
WHERE BA.author_id = A.author_id AND BA.product_id = P.product_id
    AND B.product_id = P.product_id
    AND A.author_id = 5
ORDER BY book_name
;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

BOOK_NAME       AUTHOR_NAME     PRODUCT_ID  PRICE        ISBN                PUBLISHER       PAGE_COUNT
--------------- --------------- ----------- ------------ ------------------- --------------- -----------
Coloring Book   Sophia Turner            24         7.99 978-0-888888-99-0   ColorFun House           80
Math Made Easy  Sophia Turner            19        14.50 978-0-444444-55-6   EduBooks                200
Tiny Chefs      Sophia Turner            14        13.99 978-0-222222-33-4   KidCook Press           120

  3 record(s) selected.
```

Query 2:

a) This query gets all of the products of a certain genre/category, in this case 'Educational'. It lists the products' ids, names, types and prices to be able to compare them and is ordered by product id.

b)
```
(SELECT BG.genre_name, P.product_id, P.name product_name,
    'Book' product_type, P.price
FROM BookGenre BG, Product P
WHERE BG.product_id = P.product_id
    AND BG.genre_name = 'Educational'
UNION
SELECT MG.genre_name, P.product_id, P.name product_name,
    'Movie' product_type, P.price
FROM MovieGenre MG, Product P
WHERE MG.product_id = P.product_id
    AND MG.genre_name = 'Educational')
ORDER BY product_id;
```

c) Screenshot: (Note: I used VARCHAR to make the columns thinner for the screenshot)

```
db2 => (SELECT VARCHAR(BG.genre_name,15) genre_name, P.product_id, VARCHAR(P.name,15) product_name, 'Book' product_type, P.price
FROM BookGenre BG, Product P
WHERE BG.product_id = P.product_id
    AND BG.genre_name = 'Educational'
UNION
SELECT VARCHAR(MG.genre_name,15), P.product_id, VARCHAR(P.name,15) product_name, 'Movie' type, P.price
FROM MovieGenre MG, Product P
WHERE MG.product_id = P.product_id
    AND MG.genre_name = 'Educational')
ORDER BY product_id
;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

GENRE_NAME      PRODUCT_ID  PRODUCT_NAME    PRODUCT_TYPE PRICE
--------------- ----------- --------------- ------------ ------------
Educational             13 Flying Friends  Movie              16.50
Educational             18 Nature Wonders  Movie              18.99
Educational             19 Math Made Easy  Book               14.50
Educational             24 Coloring Book   Book                7.99

  4 record(s) selected.
```

Query 3:

a) This query gets all of the orders of a certain customer, as they would be able to see it on their end, in this case customer 'Paul Kuame' with id 3. It gets all of the items bought in each order, along with the price that was paid, the quantity bought, its shipping status (and expected delivery date if not yet delivered), and whether it has been returned (and the quantity returned if so).

b)
```
SELECT O.order_id, C.first_name, C.last_name, DATE(O.order_timestamp) order_date,
    P.name product_name, OI.quantity quantity_bought, OI.price_at_purchase,
    S.status shipping_status,
    CASE WHEN S.status != 'delivered'
        THEN S.expected_delivery
    ELSE NULL
    END expected_delivery_date,
    CASE WHEN ((OI.order_id, OI.product_id) IN
                (SELECT order_id, product_id FROM "Return" R))
        THEN (CASE WHEN (OI.quantity - (SELECT SUM(R.quantity) FROM "Return" R
                                        WHERE R.order_id = OI.order_id
                                        AND R.product_id = OI.product_id) = 0)
                    THEN 'Returned'
                    ELSE 'Partially Returned' END)
    ELSE 'Not Returned'
```

```
    END returned,
    CASE WHEN ((OI.order_id, OI.product_id) IN
                (SELECT order_id, product_id FROM "Return" R))
        THEN (SELECT SUM(R.quantity) FROM "Return" R
            WHERE R.order_id = OI.order_id AND R.product_id = OI.product_id)
    ELSE NULL
    END quantity_returned
FROM "Order" O, Customer C, OrderItem OI, Product P, Shipping S
WHERE O.order_id = OI.order_id AND P.product_id = OI.product_id
    AND O.customer_id = C.customer_id AND O.shipping_id = S.shipping_id
    AND C.customer_id = 3
ORDER BY order_date;
```

c) Screenshot: (Note: I used VARCHAR to make the columns thinner for the screenshot)

```
db2 => SELECT O.order_id, VARCHAR(C.first_name,15) first_name, VARCHAR(C.last_name,15) last_name, DATE(O.order_timestamp) order_date,
    VARCHAR(P.name,20) product_name, OI.quantity quantity_bought, OI.price_at_purchase,
    VARCHAR(S.status,15) shipping_sdb2 (cont.) => tatus,
    CASE WHEN S.status != 'delivered'
        THEN S.expected_delivery
    ELSE NULL
    END expected_delivery_date,
    CASE Wdb2 (cont.) => HEN ((OI.order_id, OI.product_id) IN
            (SELECT order_id, product_id FROM "Return" R))
        THEN (CASE WHdb2 (cont.) => EN (OI.quantity - (SELECT SUM(R.quantity) FROM "Return" R
                                        WHERE R.order_iddb2 (cont.) =>  = OI.order_id
                                        AND R.product_id = OI.product_id) = 0)
    db2 (cont.) =>                THEN 'Returned'
                ELSE 'Partially Returned' END)
    ELSE 'Nodb2 (cont.) => t Returned'
    END returned,
    CASE WHEN ((OI.order_id, OI.product_id) IN
            db2 (cont.) =>   (SELECT order_id, product_id FROM "Return" R))
        THEN (SELECT SUM(R.quantity) FROM "Redb2 (cont.) => turn" R
            WHdb2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => ERE R.or
der_id = OI.order_id AND R.product_id = OI.product_id)
    ELSE NULL
    END quantity_returned
FROM "Order" O, Customer C, OrderItem OI, Product P, Shipping S
WHERE O.order_id = OI.order_id AND P.product_id = OI.product_id
    AND O.customer_id = C.customer_id AND O.shipping_id = S.shipping_id
    AND C.customer_id = 3
ORDER BY order_date
;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

ORDER_ID    FIRST_NAME      LAST_NAME       ORDER_DATE PRODUCT_NAME         QUANTITY_BOUGHT PRICE_AT_PURCHASE SHIPPING_STATUS EXPECTED_DELIVERY_DATE RETURNED           QUANTITY_RETURNED
---------- -------------- -------------- ---------- -------------------- --------------- ----------------- ---------------- ---------------------- ------------------ -----------------
         3 Paul           Kuame          02/01/2025 Love in Paris                    2             16.75 delivered        -                      Partially Returned                 1
         3 Paul           Kuame          02/01/2025 Superhero Saga                   1             19.99 delivered        -                      Not Returned                       -
        18 Paul           Kuame          07/01/2025 Mini Drone                       1             94.99 delivered        -                      Returned                           1
        18 Paul           Kuame          07/01/2025 Stuffed Bunny                    1             14.75 delivered        -                      Not Returned                       -
        18 Paul           Kuame          07/01/2025 Space Odyssey                    1             18.00 delivered        -                      Not Returned                       -
        21 Paul           Kuame          07/20/2025 Wooden Train                     3             29.99 shipped          08/13/2025             Not Returned                       -
        21 Paul           Kuame          07/20/2025 Robot Dog                        1             59.99 shipped          08/13/2025             Not Returned                       -
        21 Paul           Kuame          07/20/2025 Pride & Prejudice                2             12.50 shipped          08/13/2025             Not Returned                       -

  8 record(s) selected.
```

Query 4:

a) This query gets the information on all orders on a certain product, in this case 'Love in Paris' of product id 8. It gets the order id, the customer's name, the date it was ordered, the quantity bought, the price paid, and whether it was returned (and the quantity returned if so). This can be used to track the sales of the product, changes in price, and the return frequency.

b)
```
SELECT O.order_id, C.first_name, C.last_name, P.name product_name,
    DATE(O.order_timestamp) order_date, OI.quantity quantity_bought,
    OI.price_at_purchase,
    CASE WHEN ((OI.order_id, OI.product_id) IN
                (SELECT order_id, product_id FROM "Return" R))
        THEN (CASE WHEN (OI.quantity - (SELECT SUM(R.quantity) FROM "Return" R
                                        WHERE R.order_id = OI.order_id
                                        AND R.product_id = OI.product_id) = 0)
                    THEN 'Returned'
                    ELSE 'Partially Returned' END)
    ELSE 'Not Returned'
    END returned,
    CASE WHEN ((OI.order_id, OI.product_id) IN
                (SELECT order_id, product_id FROM "Return" R))
        THEN (SELECT SUM(R.quantity) FROM "Return" R
```

```
            WHERE R.order_id = OI.order_id AND R.product_id = OI.product_id)
    ELSE NULL
    END quantity_returned
FROM "Order" O, Customer C, OrderItem OI, Product P
WHERE O.order_id = OI.order_id AND P.product_id = OI.product_id
    AND O.customer_id = C.customer_id
    AND P.product_id = 8
ORDER BY order_date;
```

c) Screenshot: (Note: I used VARCHAR to make the columns thinner for the screenshot)

```
db2 => SELECT O.order_id, VARCHAR(C.first_name,15) first_name, VARCHAR(C.last_name,15) last_name, VARCHAR(P.name,20) product_name,
    DATE(O.order_timestamp) order_date, OI.quantity quantity_bought, OI.price_at_purchase,
    CASE WHEN ((OI.order_id, OI.product_id) IN (SELECT order_id, product_id FROM "Return" R))
        THEN (CASE WHEN (OI.quantity - (SELECT SUM(R.quantity) FROM "Return" R
                                        WHERE R.order_id = OI.order_id
                                        AND R.product_id = OI.product_id) = 0)
                  THEN 'Returned'
                  ELSE 'Partially Returned' END)
    ELSE 'Not Returned'
    END returned,
    CASE WHEN ((OI.order_id, OI.product_id) IN (SELECT order_id, product_id FROM "Return" R))
        THEN (SELECT SUM(R.quantity) FROM "Return" R
              WHERE R.order_id = OI.order_id AND R.product_id = OI.product_id)
    ELSE NULL
    END quantity_returned
FROM "Order" O, Customer C, OrderItem OI, Product P
WHERE O.orderdb2 (cont.) => _id = OI.order_id AND P.db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (c
ont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => product_id = OI.pro
duct_id
    AND O.customer_id = C.customer_id
    AND P.product_id = 8
ORDER BY order_date
;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

ORDER_ID    FIRST_NAME      LAST_NAME       PRODUCT_NAME         ORDER_DATE QUANTITY_BOUGHT PRICE_AT_PURCHASE RETURNED           QUANTITY_RETURNED
----------- --------------- --------------- -------------------- ---------- --------------- ----------------- ------------------ -----------------
          3 Paul            Kuame           Love in Paris        02/01/2025              2             16.75 Partially Returned                 1
         11 Savage          Quemby          Love in Paris        04/12/2025              1             14.25 Not Returned                       -
         19 Oneil           Jorden          Love in Paris        07/09/2025              1             14.25 Not Returned                       -

  3 record(s) selected.
```

Query 5:

a) This query gets all the shipments that aren't yet delivered. It returns the order id of the shipment, the customer's name and id, the shipping id, the shipping status, the carrier name and tracking number of the shipment, the delivery and return addresses, the order data, the expected delivery date, and the amount paid for this order.

b)
```
SELECT O.order_id, C.customer_id, C.first_name, C.last_name,
    S.shipping_id, S.status shipping_status, S.carrier_name, S.tracking_number,
    S.from_address, S.to_address,
    DATE(O.order_timestamp) order_date, S.expected_delivery expected_delivery_date,
    P.amount payment_amount
FROM Shipping S, "Order" O, Customer C, Payment P
WHERE S.shipping_id = O.shipping_id AND C.customer_id = O.customer_id
    AND P.payment_id = O.payment_id
    AND S.status != 'delivered'
ORDER BY order_date;
```

c) Screenshot: (Note: I used VARCHAR to make the columns thinner for the screenshot)

```
db2 => SELECT O.order_id, C.customer_id, VARCHAR(C.first_name,10) first_name, VARCHAR(C.last_name,10) last_name,
    S.shipping_id, VARCHAR(S.status,17) shipping_status, VARCHAR(S.carrier_name,10) carrier_name, VARCHAR(S.tracking_number,15) tracking_number,
    VARCHAR(S.from_address,46) from_address, VARCHAR(S.to_address,45) to_address,
    DATE(O.order_timestamp) order_date, S.expected_delivery expected_delivery_date,
    P.amount payment_amount
FROM Shipping S, "Order" O, Customer C, Payment P
WHEREdb2 (cont.) =>  S.shipping_id = O.shipping_id AND C.customer_id = O.customer_id
    AND P.payment_id = O.payment_id
    AND S.status != 'delivered'
ORDER BY order_date
[;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

ORDER_ID    CUSTOMER_ID FIRST_NAME LAST_NAME  SHIPPING_ID SHIPPING_STATUS   CARRIER_NAME TRACKING_NUMBER FROM_ADDRESS                                TO_ADDRESS                                ORDER_DATE EXPECTED_DELIVERY_DATE PAYMENT_AMOUNT
----------- ----------- --------- --------- ----------- ---------------- ------------ --------------- ---------------------------------------- ----------------------------------------- ---------- ------------------------ --------------
         20           5 Lopez     Maxwell            20 out for delivery  FedEx        TRK1007         4000 Rue Notre-Dame Est, Montréal, QC H1W 2K3  3100 Boulevard Pie-IX, Montréal, QC H1X 2B3  07/15/2025 07/20/2025                     95.25
         21           3 Paul      Kuame              28 shipped           CanadaPost   TRK1010         4000 Rue Notre-Dame Est, Montréal, QC H1W 2K3  890 Avenue du Parc, Montréal, QC H2W 1S9      07/20/2025 08/13/2025                    105.93

  2 record(s) selected.
```

SQL Modifications:

Mod 1:

a) Increase the price of all products that correspond to movies by 10%.

b)
```
UPDATE Product SET price = price * 1.10 WHERE product_id IN
    (SELECT product_id FROM Movie);
```

c)
```
cs421g27@winter2026-comp421:~/project2$ db2 "UPDATE Product SET price = price * 1.10 WHERE p
roduct_id IN (SELECT product_id FROM Movie)"
DB20000I  The SQL command completed successfully.
```

Mod 2:

a) This statement deletes return records associated with orders that were placed before February 1, 2025.

b)
```
DELETE FROM "Return" WHERE order_id IN
    (SELECT order_id FROM "Order" WHERE order_timestamp < '2025-02-01');
```

c)
```
cs421g27@winter2026-comp421:~/project2$ db2 "DELETE FROM \"Return\" WHERE orde
r_id IN (SELECT order_id FROM \"Order\" WHERE order_timestamp < '2025-02-01')"
DB20000I  The SQL command completed successfully.
cs421g27@winter2026-comp421:~/project2$
```

Views:

View 1:

a) This view shows all customers' order history without payment or shipping information, so as to hide sensitive data.

b)
```
CREATE VIEW ORDERHISTORY AS
    SELECT order_id, customer_id, order_timestamp
    FROM "Order";
```

c)
```
db2 => CREATE VIEW ORDERHISTORY AS SELECT order_id, customer_id, order_timestamp FROM "Order";
DB20000I  The SQL command completed successfully.
```

d)
```
db2 => SELECT order_id, customer_id, order_timestamp FROM ORDERHISTORY FETCH FIRST 5 ROWS ONLY;

ORDER_ID    CUSTOMER_ID ORDER_TIMESTAMP
----------- ----------- --------------------------
          1           1 2025-01-10-10.00.00.000000
          2           2 2025-01-15-12.30.00.000000
          3           3 2025-02-01-09.15.00.000000
          4           4 2025-02-10-14.20.00.000000
          5           5 2025-02-18-16.45.00.000000

  5 record(s) selected.
```

e)
```
db2 => INSERT INTO ORDERHISTORY VALUES (123, 1, '2026-01-01-00.00.00.000000');
DB20000I  The SQL command completed successfully.
```

f) The view is based on a single table “Order” and only includes 3 columns from the base table (order_id, customer_id, order_timestamp). The `INSERT` was successful because DB2 allows inserts through simple views if all of the `NOT NULL` columns in the underlying table are included in the view or have default values and the values provided for insertion in the view are valid domain values (match column types and constraints).
In our case, all the mandatory values (order_id, customer_id, and order_timestamp) were provided, so the insertion worked, and this row will appear in the base table.

View 2:

a) This view shows a book catalog, including product ID, name, price, publisher, and ISBN.

b)
```
CREATE VIEW BookCatalog AS
    SELECT p.product_id, p.name, P.price, b.publisher, b.isbn
    FROM PRODUCT p JOIN BOOK b ON p.product_id = b.product_id;
```

c)
```
db2 => CREATE VIEW BookCatalog AS SELECT p.product_id, p.name, p.price, b.publisher, b.isbn
 FROM PRODUCT p JOIN BOOK b ON p.product_id = b.product_id;
 DB20000I  The SQL command completed successfully.
```

d)
```
db2 => SELECT product_id, CAST(name AS VARCHAR(30)), price, CAST(publisher AS VARCHAR(30)), isbn FROM BookCatalog FETCH FIRST 5 ROWS ONLY;

PRODUCT_ID  2                              PRICE       4                              ISBN
----------- ------------------------------ ----------- ------------------------------ --------------------
          2 Dragon Quest                         15.50 Fantasy Press                  978-0-123456-47-2
          5 Space Odyssey                        18.00 SciFi House                    978-0-987654-32-1
          8 Love in Paris                        14.25 Romance Co.                    978-1-234567-89-0
         10 Pride & Prejudice                    12.50 Classic Books                  978-0-111111-22-3
         14 Tiny Chefs                           13.99 KidCook Press                  978-0-222222-33-4

  5 record(s) selected.
```

e)
```
db2 => INSERT INTO BookCatalog VALUES(999, 'New Book', 29.99, 'Publisher', '999999999');
 DB21034E  The command was processed as an SQL statement because it was not a
 valid Command Line Processor command.  During SQL processing it returned:
 SQL0150N  The target fullselect, view, typed table, materialized query table,
 range-clustered table, or staging table in the INSERT, DELETE, UPDATE, MERGE,
 or TRUNCATE statement is a target for which the requested operation is not
 permitted.  SQLSTATE=42807
```

f) The `INSERT` failed with error code `SQL0150N` because the view involves joining two tables (Product and Book). DB2 doesn’t allow inserting into views that are defined by join operations.

Check Constraints:

Check 1:

a) This constraint ensures that the price of a product must always be positive. A product with a price equal to 0 or less than zero would be invalid in an online bookstore because an item cannot be sold for free or for a negative amount.

b)
```
ALTER TABLE Product ADD CONSTRAINT positive_price CHECK (price > 0);
```

c)
```
db2 => db2 => db2 => db2 => db2 => db2 (cont.) => db2 (cont.) => ALTER TABLE Product ADD CONSTRAINT positive_price CHECK (price > 0)
DB20000I  The SQL command completed successfully.
```

d)

```
cs421g27@winter2026-comp421:~/project2$ db2 "INSERT INTO Product VALUES (999, 'test', 'test', 10, -5)"
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0545N  The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G27.PRODUCT.POSITIVE_PRICE".
SQLSTATE=23513
cs421g27@winter2026-comp421:~/project2$
```

Check 2:

a) This constraint ensures that the rating of a movie must be one of the valid MPAA ratings: G, PG, PG-13, R, NC-17 or `NULL` if the movie is not rated.

b)
```
ALTER TABLE Movie ADD CONSTRAINT valid_movie_rating
  CHECK (rating IN ('G', 'PG', 'PG-13', 'R', 'NC-17') OR rating IS NULL);
```

c)

```
cs421g27@winter2026-comp421:~/project2$ db2 "ALTER TABLE Movie ADD CONSTRAINT valid_movie_rating CHECK (rating IN ('G', 'PG', 'PG-13', 'R') OR rating IS
 NULL)"
DB20000I  The SQL command completed successfully.
cs421g27@winter2026-comp421:~/project2$
```

d)

```
cs421g27@winter2026-comp421:~/project2$ db2 "INSERT INTO Movie VALUES (1, 'INVALID', 120, 'director 1', '2025-01-01')"
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0545N  The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G27.MOVIE.VALID_MOVIE_RATING".
SQLSTATE=23513
cs421g27@winter2026-comp421:~/project2$
```

Creativity:

We did an Advanced SQL Feature:

a) This query uses a window function (rank) to rank products by the total revenue they've generated throughout all orders, from highest to lowest.

b) Query:
```
SELECT p.product_id, p.name, SUM(oi.quantity * oi.price_at_purchase)
    AS total_revenue, RANK() OVER (ORDER BY SUM(oi.quantity *
    oi.price_at_purchase) DESC) AS revenue_rank
FROM Product p JOIN OrderItem oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name;
```

c)

```
db2 => SELECT p.product_id, p.name, SUM(oi.quantity * oi.price_at_purchase) AS total_revenue, RANK() OVER (ORDER BY SUM(oi.quantity
* oi.price_at_purchase) DESC) AS revenue_rank FROM Product p JOIN OrderItem oi ON p.product_id = oi.product_id GROUP BY p.product_id
, p.name;

PRODUCT_ID  NAME                                               TOTAL_REVENUE                     REVENUE_RANK
----------- -------------------------------------------------- --------------------------------- --------------------
         21 Robot Dog                                                                     119.98                    1
          1 Wooden Train                                                                  119.95                    2
         29 Mini Drone                                                                     94.99                    3
          5 Space Odyssey                                                                  92.97                    4
         10 Pride & Prejudice                                                              89.50                    5
          8 Love in Paris                                                                  62.00                    6
         20 Block Builder                                                                  59.99                    7
          2 Dragon Quest                                                                   59.97                    8
         14 Tiny Chefs                                                                     56.25                    9
         17 The Secret Diary                                                               45.98                   10
         24 Coloring Book                                                                  40.00                   11
         11 Cuddly Bear                                                                    34.99                   12
         19 Math Made Easy                                                                 33.98                   13
          7 Candy Kingdom                                                                  30.00                   14
         27 Science Lab Kit                                                                30.00                   14
          4 Math Puzzle                                                                    29.99                   16
         15 Junior Scientist                                                               27.99                   17
         26 Building Blocks Deluxe                                                         25.00                   18
          3 Superhero Saga                                                                 19.99                   19
         22 Magic Puzzle                                                                   19.99                   19
         13 Flying Friends                                                                 18.50                   21
         25 High School Musical 2                                                          14.99                   22
         28 Stuffed Bunny                                                                  14.75                   23
         18 Nature Wonders                                                                 14.25                   24
          9 Nightmare Street                                                               13.50                   25
         23 Kids Tablet                                                                    12.49                   26
         12 Brain Twister                                                                  11.25                   27
         30 Puzzle Cube Advanced                                                           10.50                   28
          6 Speed Racer                                                                     9.99                   29
         16 Captain Bold                                                                    8.99                   30

  30 record(s) selected.
```

Member Contributions:

We had one meeting to split up the work, then we discussed each other's work through text. Steven did Task 3 and started Task 4. Mujtaba finished Task 4 and did Tasks 6 and 8. Alice did Task 5. Eleni did Tasks 7 and 9.